

Universidad Nacional de Colombia

Facultad de Ingeniería — Ingeniería de Sistemas
Sistemas Operativos

Informe de Elaboración

Práctica 2: Sistema Cliente-Servidor IMDb

Integrantes:

Daniel Alfonso Cely Infante
Maria Catalina Rodriguez Cardona
Yeral Francesco Pacherre Trujillo

2026

1. Descripción General

Se implementó un sistema cliente-servidor para búsqueda y gestión de películas y series de la base de datos IMDb. El servidor gestiona un archivo binario en disco (películas.bin) y atiende hasta 32 clientes simultáneos mediante hilos POSIX. El cliente presenta un menú interactivo y se comunica con el servidor a través de sockets TCP en el puerto 3535.

La arquitectura reutiliza los módulos de indexación Hash.c, Busqueda.c y Conversion_Bin.c desarrollados en la Práctica 1, adaptando la capa de comunicación de memoria compartida a sockets TCP.

2. Descripción de Funciones

2.1 p2-server.c

Función	Descripción
<code>main()</code>	Inicializa los 3 mutexes, crea el socket, lo enlaza al puerto 3535 con <code>bind()</code> , lo pone en escucha con <code>listen()</code> y entra en un loop infinito aceptando conexiones con <code>accept()</code> . Por cada cliente crea un hilo con <code>pthread_create()</code> y llama a <code>pthread_detach()</code> para que se limpie automáticamente.
<code>handle_client()</code>	Función ejecutada por cada hilo de cliente. Obtiene la IP del cliente con <code>getpeername()</code> , entra en un loop que recibe queries con <code>recv()</code> y responde según el tipo: SEARCH llama a las funciones de búsqueda y hace <code>send()</code> de vuelta; ADD_MOVIE inserta la película; EXIT cierra la conexión y decrementa el contador.
<code>escribir_log()</code>	Registra cada operación en server.log con formato [YYYYMMDDTHHMMSS] Cliente [IP] [accion]. Usa <code>mutex_log</code> para evitar que múltiples hilos escriban al mismo tiempo y corrompan el archivo.

2.2 p2-client.c

Función	Descripción
<code>main()</code>	Crea el socket con <code>socket()</code> , se conecta al servidor con <code>connect()</code> y entra en el loop del menú. Llama a <code>mostrarMenu()</code> y despacha la opción del usuario al case correspondiente.

<code>buscarPorTitulo()</code>	Lee el título del usuario, llena <code>response.query</code> con <code>searchCriteria=SEARCH</code> y sin filtros, hace <code>send()</code> al servidor y espera <code>recv()</code> con la respuesta. Llama a <code>mostrarResultado()</code> .
<code>buscarConFiltros()</code>	Igual que <code>buscarPorTitulo()</code> pero además pide tipo, año y género al usuario. Los filtros en blanco se dejan como cadena vacía o -1 para que el servidor los ignore.
<code>agregarPelicula()</code>	Pide todos los campos de la película al usuario con validación de campos obligatorios, llena <code>response.movie</code> , establece <code>searchCriteria=ADD_MOVIE</code> , hace <code>send()</code> y espera <code>recv()</code> de confirmación.
<code>mostrarResultado()</code>	Muestra los datos de la película encontrada. Si <code>found==0</code> ofrece agregar la película llamando a <code>agregarPelicula()</code> .
<code>salir()</code>	Envía una query con <code>searchCriteria=EXIT</code> al servidor para cerrar el hilo limpiamente, muestra el mensaje de despedida.
<code>esperarTecla()</code>	Pone el terminal en modo raw desactivando <code>ICANON</code> y <code>ECHO</code> con <code>tcsetattr()</code> , captura cualquier tecla con <code>getchar()</code> y restaura la configuración original.

2.3 Hash.c

Función	Descripción
<code>calcular_hash()</code>	Implementa el algoritmo djb2: $\text{hash} = \text{hash} * 33 + c$ para cada carácter. Retorna $\text{hash} \% \text{HASH_SIZE}$ (999) para obtener el índice del bucket.
<code>inicializar_hash_table()</code>	Rellena el arreglo <code>hash_table[999]</code> con -1 para indicar que cada posición está vacía.
<code>guardar_hash_en_binario()</code>	Serializa el arreglo <code>hash_table</code> completo en <code>hash.bin</code> con <code>fwrite</code> .
<code>cargar_hash_desde_binario()</code>	Carga <code>hash.bin</code> a RAM con <code>fread</code> al iniciar el servidor.

2.4 Busqueda.c

Función	Descripción
<code>buscar_por_nombre()</code>	Calcula el hash del nombre buscado, obtiene el offset inicial del bucket y recorre la cadena de <code>next_offset</code> en <code>peliculas.bin</code> con <code>fseek/fread</code> hasta encontrar coincidencia exacta por <code>primaryTitle</code> .
<code>buscar_por_filtros()</code>	Igual que <code>buscar_por_nombre()</code> pero verifica todos los filtros activos (<code>titleType</code> , <code>startYear</code> , <code>genres</code>). Un campo vacío o -1 se

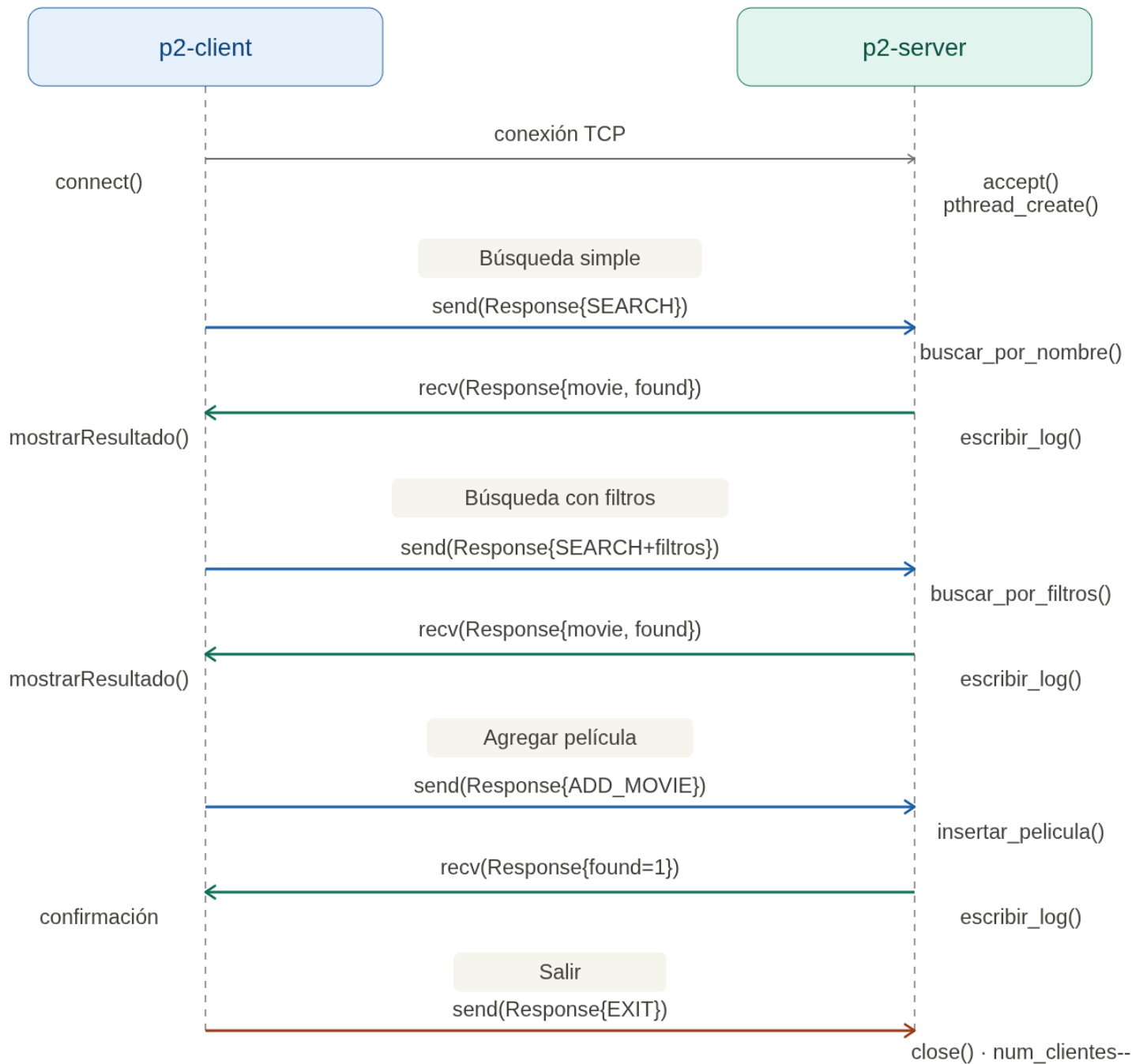
	ignora. Retorna la primera película que coincida con todos los filtros activos.
--	---

2.5 Conversion_Bin.c

Función	Descripción
<code>leer_linea_tsv()</code>	Lee una línea del TSV con <code>fgets</code> y la parsea campo a campo con <code>strtok</code> usando tabulador como delimitador. Copia los valores a la struct <code>Movie</code> con <code>strncpy</code> asegurando el null terminator.
<code>escribir_pelicula_bin()</code>	Obtiene el offset actual con <code>ftell</code> , encadena la nueva película en el bucket correspondiente actualizando <code>next_offset</code> y <code>hash_table</code> , y escribe la struct con <code>fwrite</code> .
<code>convertir_tsv_a_binario()</code>	Loop principal de conversión: reserva una struct <code>Movie</code> con <code>malloc</code> , lee cada línea del TSV y la escribe en binario. Usa un único <code>malloc/free</code> para todo el proceso.
<code>insertar_pelicula_en_binario()</code>	Abre <code>peliculas.bin</code> en modo <code>append</code> , ubica el final con <code>fseek</code> , encadena la nueva película en el hash y actualiza <code>hash.bin</code> en disco.

3. Diagrama de Comunicaciones

El protocolo de comunicación se basa en el intercambio de estructuras Response completas entre cliente y servidor. El cliente siempre inicia con send() y el servidor siempre responde con send() excepto en EXIT.

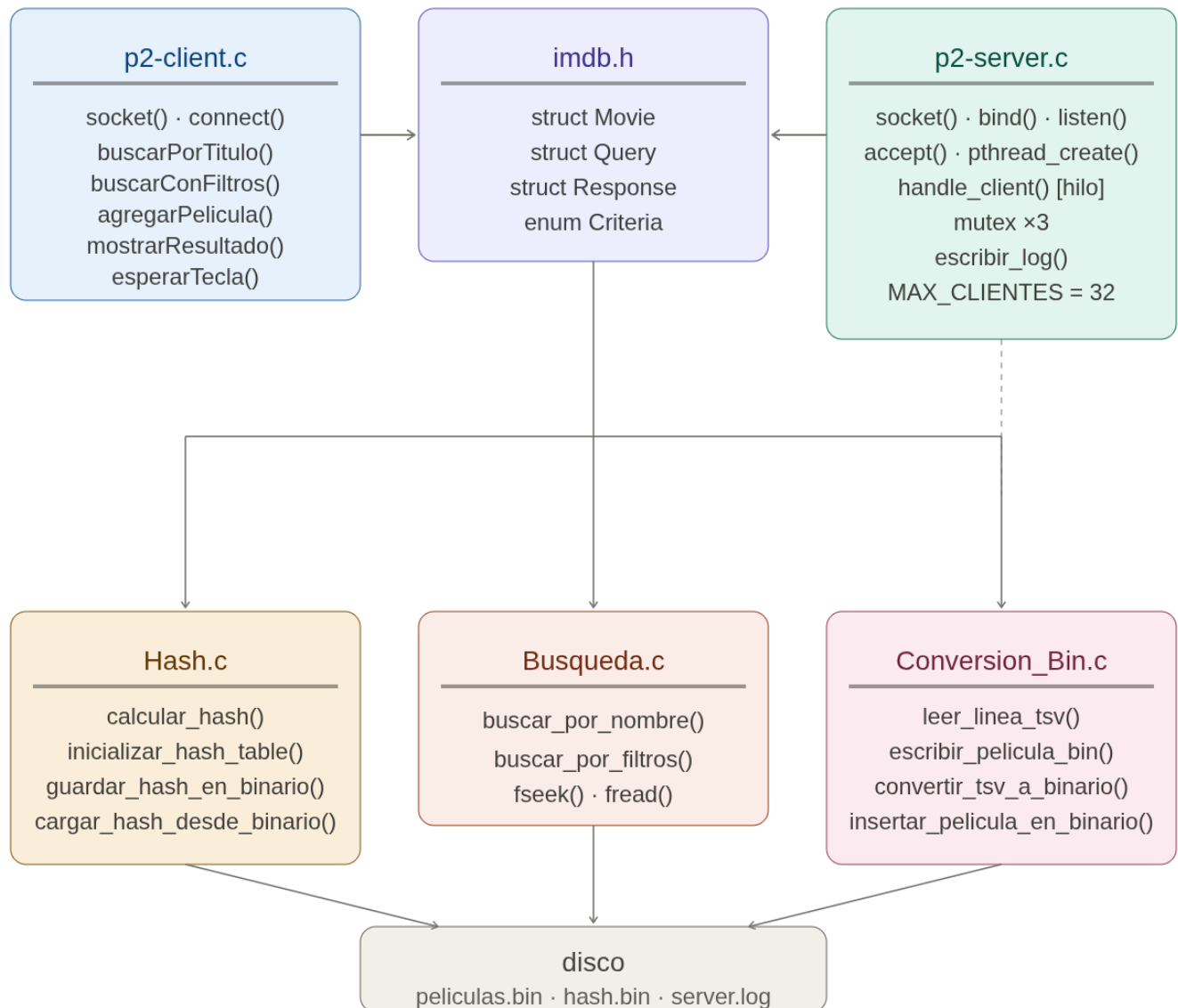


Estructura Response: contiene Query (searchCriteria, primaryTitle, filtros), Movie (resultado), found (0/1) y search_time_ms.

Puerto: 3535 TCP. **Protocolo:** IPv4. **Dirección:** 127.0.0.1 (localhost).

4. Diagrama de Bloques

El sistema está compuesto por los siguientes módulos. El servidor depende de todos los módulos de soporte; el cliente solo depende de imdb.h para los tipos compartidos.



4.1 Dependencias entre módulos

Módulo	Depende de
--------	------------

p2-server.c	imdb.h, Hash.c, Busqueda.c, Conversion_Bin.c
p2-client.c	imdb.h
Busqueda.c	imdb.h, Hash.c (hash_table global)
Conversion_Bin.c	imdb.h, Hash.c (hash_table global)
Hash.c	imdb.h

5. Decisiones de Diseño

5.1 Protocolo de comunicación

Se definió una única estructura Response como unidad de comunicación. El cliente la llena con la Query antes de hacer send() y el servidor la rellena con el resultado antes de hacer send() de vuelta. Esto simplifica el protocolo: siempre es un send seguido de un recv, equivalente al sem_post/sem_wait de la práctica anterior.

5.2 Un hilo por cliente con pthread_detach

Por cada conexión aceptada se lanza un hilo con pthread_create() y se desvincula con pthread_detach(). Esto significa que el hilo se limpia automáticamente al terminar sin necesidad de pthread_join(), evitando que el hilo principal se bloquee esperando.

5.3 Tres mutexes independientes

Se usaron tres mutexes separados en lugar de uno global para maximizar la concurrencia:

- mutex_archivo: protege fseek/fread/fwrite sobre peliculas.bin. Necesario porque FILE* no es thread-safe.
- mutex_log: protege fprintf sobre server.log. Sin él, líneas de múltiples hilos se mezclarían.
- mutex_clientes: protege num_clientes. Necesario porque el hilo principal incrementa y los hilos de cliente decrementan concurrentemente.

5.4 Datos en disco, hash en RAM

Con ~12 millones de registros, almacenar todo en RAM requeriría varios GB. La solución adoptada mantiene solo un arreglo de 999 long (~8 KB) en RAM como tabla hash. Las películas residen en peliculas.bin y se acceden con fseek/fread. Las colisiones se encadenan mediante el campo next_offset dentro de cada struct Movie en disco.

5.5 malloc para el fd de cada cliente

El fd del socket cliente se aloja con malloc antes de pasarlo al hilo. Si se pasara la dirección de la variable local sockfdc, el loop del main la sobrescribiría con el siguiente cliente antes de que el hilo la leyera. El hilo hace free(arg) inmediatamente tras leer el valor.

6. Dificultades Encontradas

- Mezcla de response-> y response. en handle_client: al declarar Response como variable local (no puntero) se deben usar puntos, no flechas. El error solo aparece en compilación.
- recv fuera del while: inicialmente el servidor recibía una sola query por cliente y cerraba. Se corrigió metiendo el recv dentro del loop para atender múltiples consultas por sesión.
- break no sale del while: en el case EXIT, break solo sale del switch. Se usó una variable corriendo=0 para controlar el while exterior.
- Address already in use: al matar el servidor con Ctrl+C el puerto queda en TIME_WAIT. Se mitiga esperando unos segundos antes de reiniciar.
- include de linux/time.h: conflictuaba con sys/select.h. Se reemplazó por time.h estándar.
- hash_table undeclared: Conversion_Bin.c usa hash_table definido en Hash.c. Se resolvió agregando extern long hash_table[HASH_SIZE] en imdb.h.